

一文了解GBF

版本	修订人	日期	内容
v0.1	王亚雷	2023.4.26	初版
v0.2	王亚雷	2023.6.1	更新引用文章、链接、群二维码
v1.0	王亚雷	2024年05月08日	增加脚手架快速接入说明
v1.2	王亚雷	2024年05月27日	更新文档部分说明和总结

1. 快速接入



复制代码

```
mvn archetype:generate -DarchetypeGroupId=com.amap.gbf.demo -
DarchetypeArtifactId=gbf-demo-archetype -DarchetypeVersion=2.0.0-SNAPSHOT
```

通过上述maven脚手架命令一键创建自己的GBF服务，该示例服务已包含：

- 完整的GBF服务示例
 - gbf框架配置
 - gbf各层编写示例
 - gbf组件使用示例
 - 使用说明注释
 - 测试用例及mock示例
- 额外的部署优化
 - VipServer平滑上下线
 - hsf上线预热
 - http接口启动预热（默认关闭）
 - JDK8的G1垃圾回收器选项（默认开启）
 - 最新稳定版pandora-boot（2023-07-stable-final）
 - 其他部署优化脚本
- 开箱即用的服务配置
 - logback异步日志配置
 - status.taobao健康检测接口
 - 其他服务配置

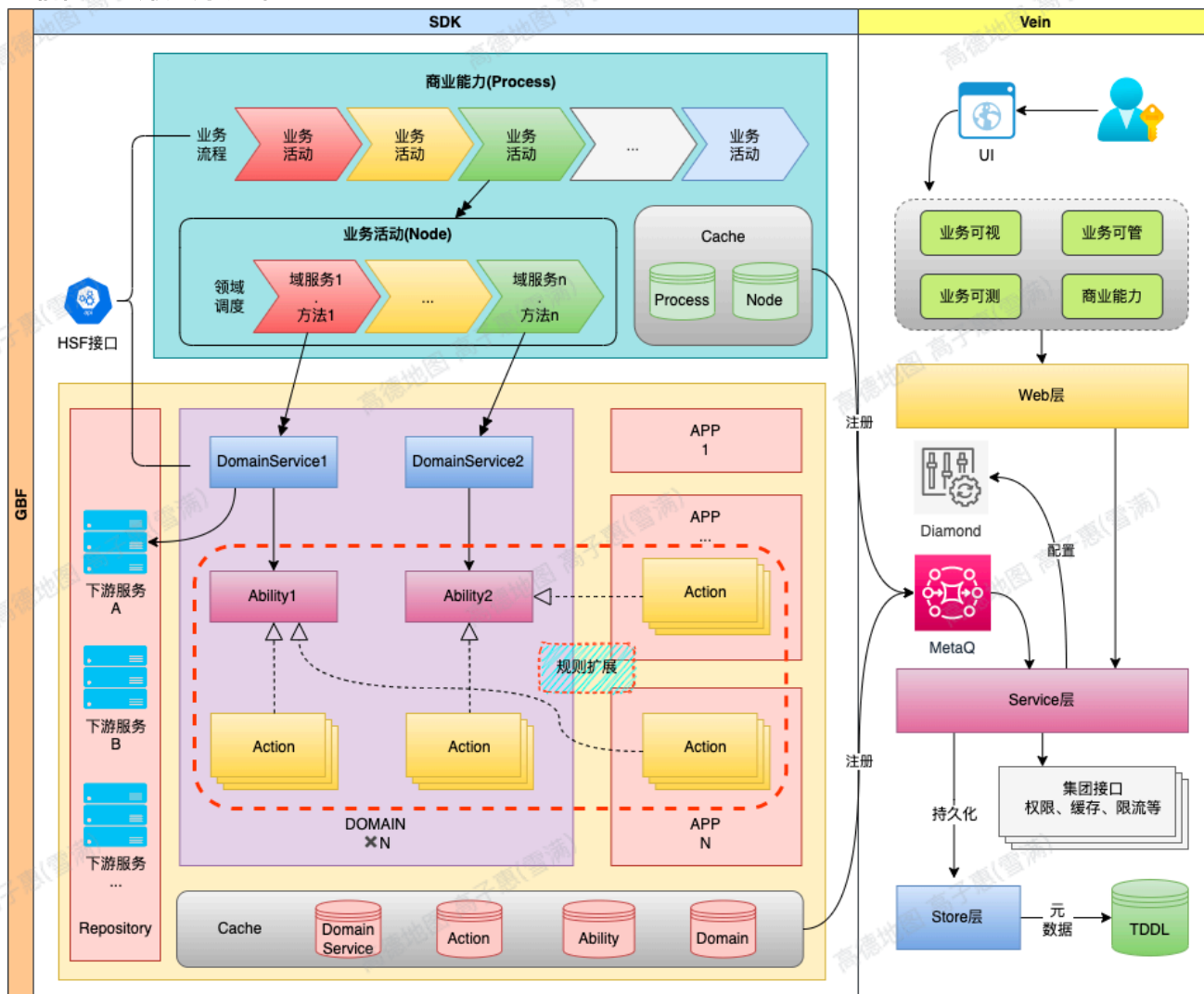
2. 概述

2.1 简介

GBF主要由两个项目构成

1. 业务服务接入需要引入的依赖SDK——GBF框架
2. 业务服务接入后配套的管控、可视化网站——业脉平台

它们间的关联关系如下



- 定位：开发框架+平台框架
- 开发框架：规范开发流程，提升开发效率
- 平台框架：帮助各域快速建立平台能力

2.2 GBF

[GBF分层设计模型](#)

平台研发目标：

1. 流程统一、复用（现状：多套流程统一，代码、功能复用）

怎么做？

分层：平台聚焦在业务流程的抽象和统一，将影响业务流程的业务规则（如根据不同渠道商请求内外不一样）部分留在平台，将业务规则类（如库存<10 展示库存紧张、消息推送通知等）的需求释放到行业能力开发团队；

GBF提供分层的项目框架和研发规约，并向对应研发团队进行宣导；

GBF提供高效的研发工具供业平台团队迭代流程；

2. 平台与外部团队合作高效：系统解耦、各司其职

怎么做？

GBF提供代码模块层面的分层，GBF提供平台对业务定制字段有兼容能力的编程模式（痛点：添加、删除业务字段），避免动一个字段从前改到后，GBF联动各个模块之间的依赖和调度执行

行业研发目标：

1. 高效的交付：复用、灵活

复用：业务隔离、复用代码、复用流程

通过[bizCode, scenarioID]完成业务流程的执行

灵活：规则配置

对流程节点具备定制的规则处理

2.3 业脉

00-业脉系统 (vein)

承接GBF在开发侧的抽象定义和业务逻辑，将其在前端平台进行展示，研发目标如下：

1. 可视能力

a. 业务流程：面向业务

b. 链路视图：面向全体

c. 分层数据：面向开发

2. 可测能力（前端建设中）

a. 服务HSF、http接口

b. GBF任意层（Process、Node、DomainService、ability、action）

c. Fetcher客户端（http、hsf）

d. 规则扩展

3. 监控能力

a. 业务身份维度的请求监控

b. 流程、节点维度的请求监控

c. 业务定制指标的记录监控

4. 管理能力

a. 接入平台管理

b. 下游访问管控

2.4 历程

2.4.1 过往

- 2022年2月 业务开发框架调研选型，在MDF基础上研发：[00-业务开发框架调研](#)
- 2022年3月 将原项目开发语言迁移java，完成框架概念的统一
- 2022年4月 核心功能完成，说明文档完成
- 2022年5月 可视化业脉上线[00-业脉系统 \(vein\)](#)
- 2022年6月 线上调试功能[06-业脉可测能力方案](#)
- 2022年7月 第一个业务服务改造完成
- 2022年12月 重新解读，形成了新的GBF分层及模型标准（Process、DomainService、扩展）
[GBF分层设计及模型](#)
- 2023年1月 新版使用手册[GBF使用手册-20230103](#)
- 2023年2月 [GBF业务落地指导小白书](#)、[US示例项目](#)
- 2023年3月 US业务改造落地（portal）、新版GBF核心功能
 - 新增分层：ComplexNode、DomainService、PlatformAbility、DomainAbility、Repo
 - 规则扩展：Action扩展、注解扩展
 - 身份路由升级改造、业务身份多级支持、Process后处理
- 2023年4月 新版业脉系统
- 2023年5月 框架组件（fetcher、threadpool、Extension等）
- 2023年8月 框架流控能力
- 2023年11月 框架缓存组件
- ...

2.4.2 当前

- Us侧基于GBF改造的核心服务us-business上线并稳定运行一年多
 - 经历了多个节日和大促考验
 - 核心接口（搜索、详情、portal等）基本完成迁移并稳定运行
 - 承担峰值QPS27w+
 - 通过合理的架构设计，实现US系统50%的资源节省，50%的发布效率的提升，应用链路缩短30%
- 组织中其他团队稳步推进老服务向GBF迁移改造ing
- ...

2.4.3 后续

- 剩余业务迁移改造上线
- 逐步下线原有的服务
- 进一步迭代优化
- ...

3. 核心思想

- 抽象复用

- 业务流程统一复用
- 业务规则扩展定制
- 业务领域统一
- 业务代码复用

- 解耦隔离

- 平台与业务分离
- 领域与领域隔离
- 业务与业务解耦
- 系统与系统解耦
- 管理域与运行域分离

- 研发提效

- 团队间职责边界清晰解耦
- 研发与产品业务认知统一
- 平台与领域具备全链路视图
- 规则扩展方便高效灵活
- 服务任意位置可测可调
- 下游访问统一收容管控
- 业务指标灵活定制监控

GBF是基于DDD思想抽象出来的产品，所以如上所示，我们所做的事也是围绕DDD理论的核心价值而展开的：

- 统一语言：不同团队形成通用（业务）语言，减少偏差
- 分而治之：垂直划分，明确边界，降低耦合，提升效率
- 分离关注：水平划分，清晰结构，逻辑分离（业务、技术），模型分离（业务、领域、储存），降低复杂度

下面我们从两个方面来解读上述核心思想

3.1 SDK

首先是SDK，其作为GBF核心理念的实际载体，直接承担了抽象复用的全部、解耦隔离的大部分职责；

它的核心思想是分层、规范、统一、定制；

而分层又是这一切实现的基础，因此我们从不同的分层来看SDK的架构。

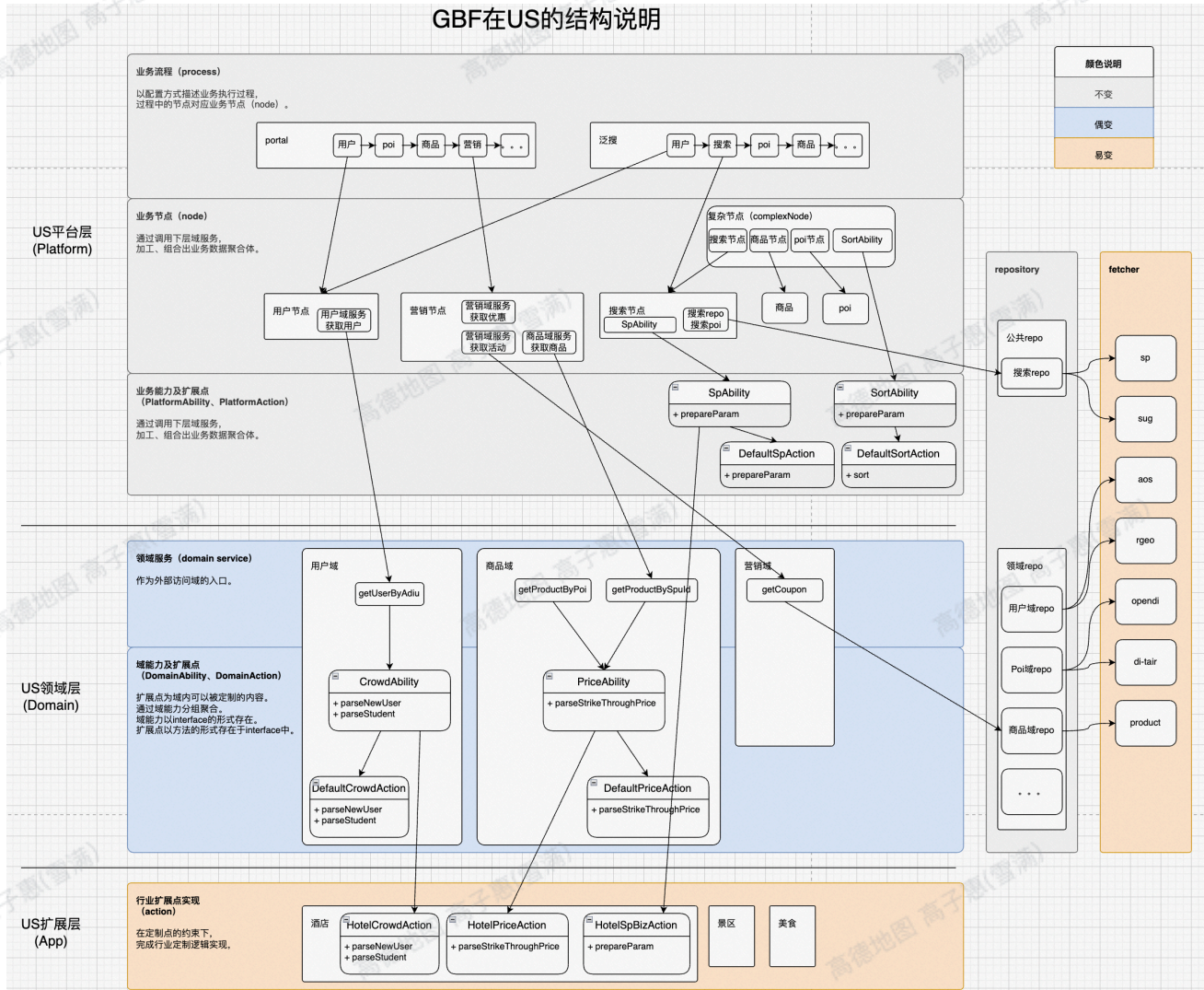
3.1.1 逻辑分层

GBF运用DDD的思想，对业务分为了如下几类，实现了技术与业务的分离：

- Process：业务流程编排层
- NodeService：流程节点服务
- DomainService：领域服务
- Ability：扩展点声明
 - Action：扩展点实现
- Repo：访问下游数仓

- Fetcher：访问下游服务的客户端

综上所述，下面用GBF在US的结构进行分层逻辑的展示：

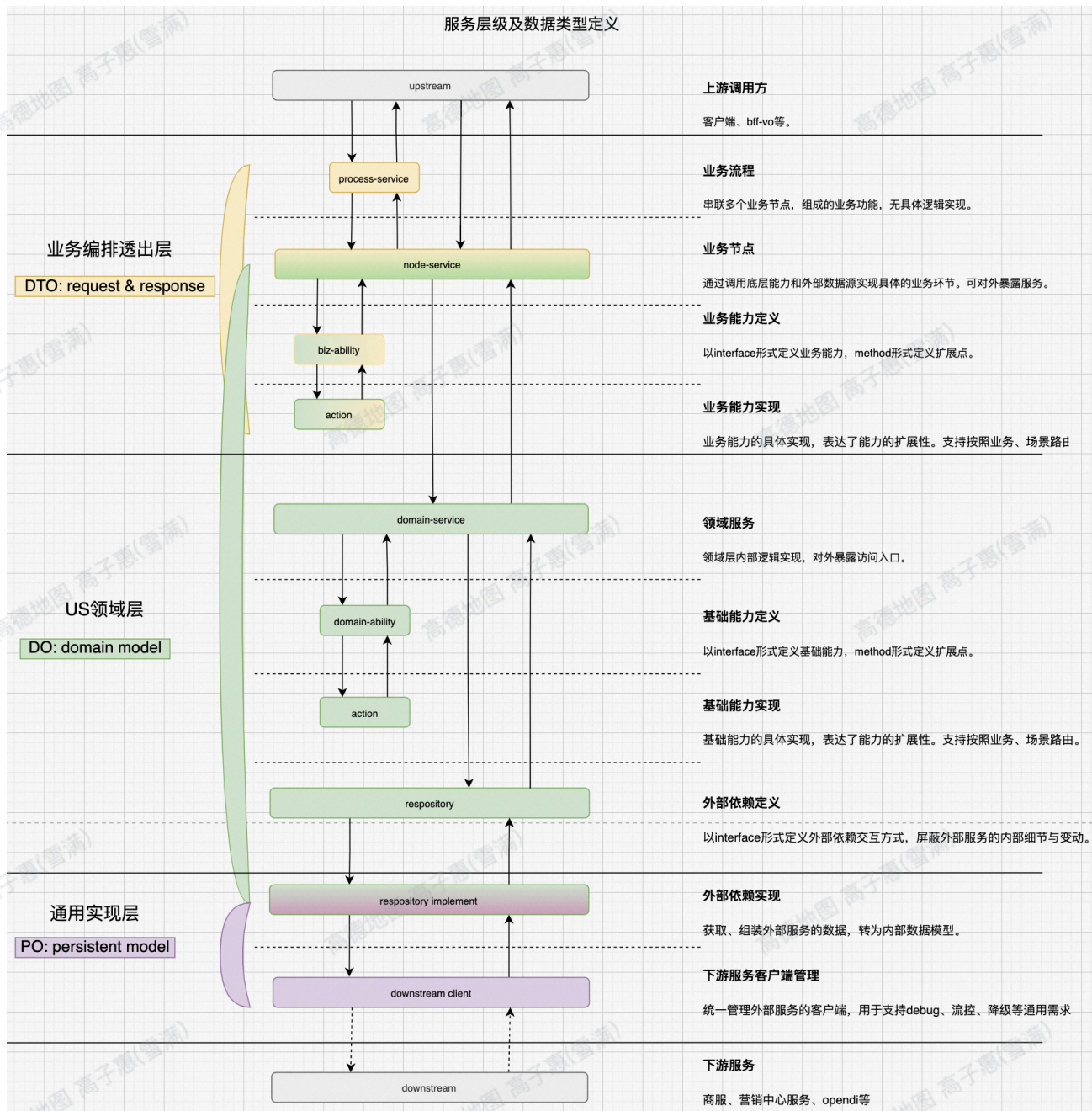


接入GBF后的框架分层清晰合理，同时实现了动静分离，做到了：

1. 业务流程统一
2. 业务规则灵活定制
3. 领域沉淀复用
4. 平台与业务水平分离
5. 领域间垂直隔离
6. 业务、系统间解耦
7. 下游访问统一收容管控

3.1.2 数据分层

逻辑分层以后，相应地我们也需要将各层的数据进行梳理，将数据也进行分层：

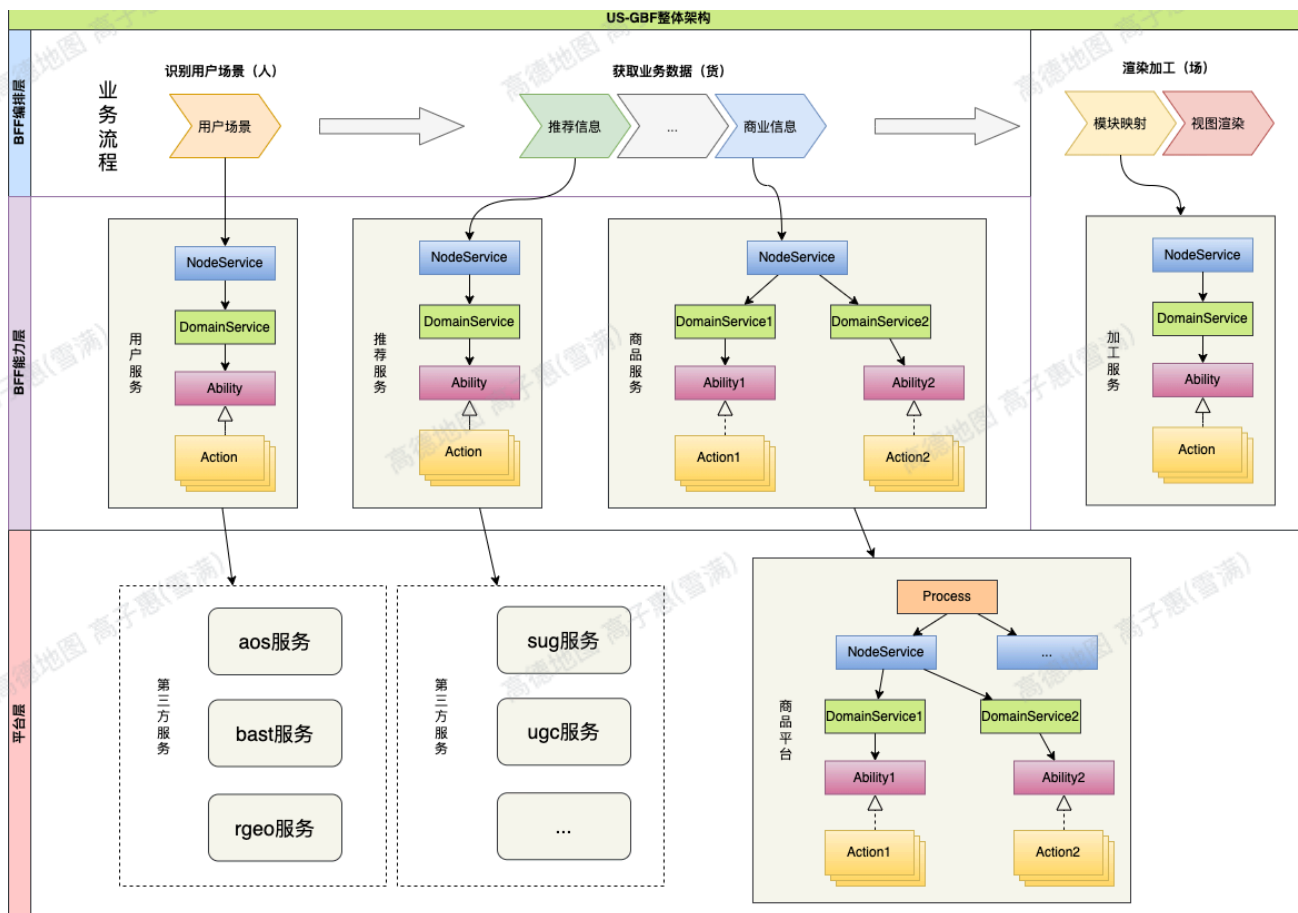


如上所示，我们将数据主要分为三大类，实现了模型的分离：

- DTO：不同业务主题场景所加工透出的业务数据
- DO：划分出的不同领域的域模型数据
- PO：访问下游获取的持久化数据

3.1.3 系统分层

除此之外，当我们的系统变得复杂，其他因素导致需将某个域或平台单独部署时；我们可以像下面这样将系统分层，来实现分而治之的目的：



在GBF中流程、节点作为平台的透出接口，域服务作为领域的透出接口；

在必要的时候可以以它们为边界，进行系统级的拆分；

分层后的单应用业务将变得单纯简洁，同时平台与业务、业务间、领域间、系统间的物理耦合度也将降低。

3.2 业脉

当然我们有了面向业务开发的SDK还是不够的；

1. 首先，开发同学从原来相对灵活自由的技术体系转换到GBF后，会需要对DDD思想有个学习的过程，同时也会面临框架的规范性所带来的桎梏；对我们的开发同学来说短期内研发难度会有所上升；
2. 其次，SDK作为接入研发环节的框架，它更多的偏向于开发和运行过程的规范约束，对接入它的业务的解释性还比较局限；
3. 最后，作为一个具有完备分层，且负责了整个业务执行调度过程的框架，它在研发调试、稳定性管控等方面也具有很好的实现优势；

所以，我们在研发了GBF的SDK项目的同时，也配套规划了基于它的前端业脉平台；

初衷是基于**SDK**集成于业务服务的优势，结合我们所凝聚的**DDD**思想，对所接入的业务进行提效赋能；

用大白话说就是让GBF变得更高效好用。

3.2.1 业务解释

面对复杂项目，由于合作方多，上下游配合周期长，沟通过程往往会产生很大偏差，增加项目的成本和风险。

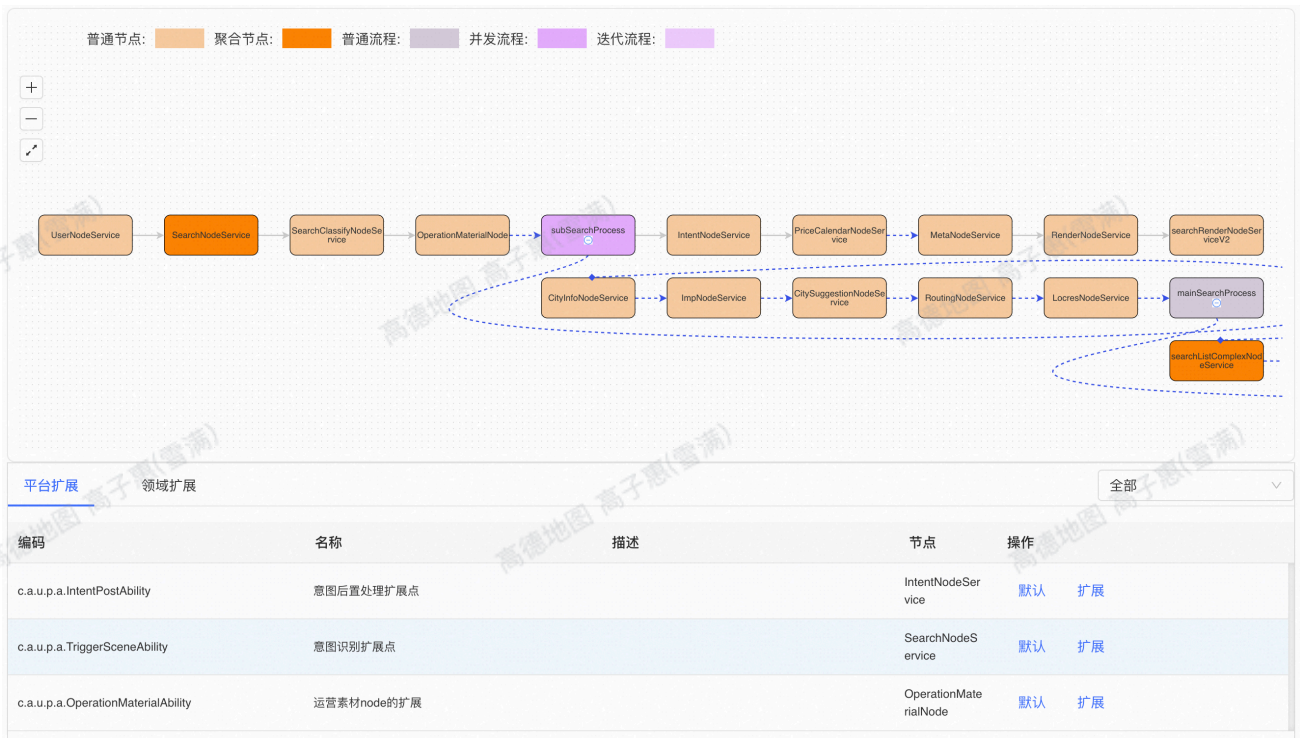


领域思想的核心价值之一就是带来统一的语言，它主要体现在

1. 产品与研发间，业务与技术语言的统一
2. 研发与研发间，不同上下游技术语言的统一
3. 研发与测试间，迭代变更及影响范围的描述语言统一
4. 团队与团队间，沟通协作语言的统一

而统一的语言就是解决协作沟通问题，消除过程差异，提升研发效率的关键所在；

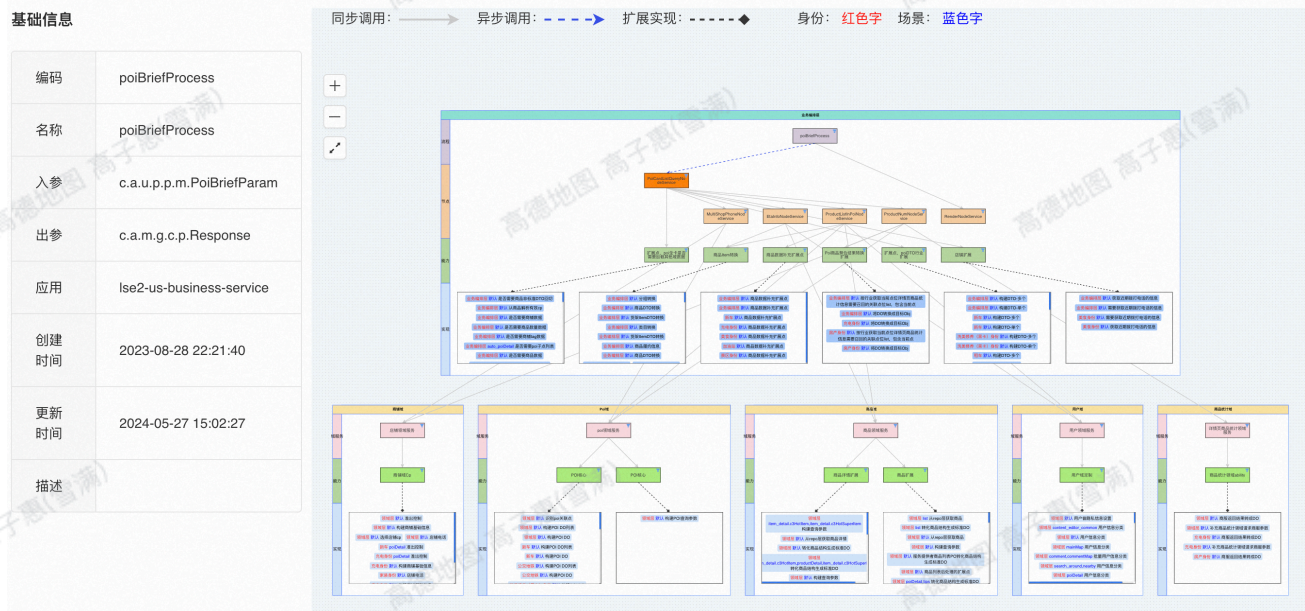
业脉在可视化方面基于SDK接入业务服务的优势，将SDK所上报的业务数据进行了可视化解析呈现；如下是业脉平台的业务流程页面：



1) 它表述的是SDK分层中最顶层的**Process（业务流程）**；通过这个界面

- 可以非常直观地了解到后台服务的业务实现逻辑；
- 可以非常全面地了解到后台业务的可定制规则扩展；

进一步，我们为想要更加全面了解某个业务流程细节的研发同学提供了链路拓扑图：



2) 上图在所有数据（process、node、DomainService、Ability...）的详情弹窗中均有展现，它将所属元数据在整个服务中的出入链路完整地展现了出来，借助于此我们可以清晰地看见

- 当前数据的上游调用方、下游调用方、改动影响范围
- 某个业务扩展的身份、场景等路由信息
- 这些节点所属的平台或划分的领域

3) 另外，我们也将在今后的将来对框架中的模型进行统一收容，每个接入GBF的平台和服务都能通过代码中注解配置的方式，生成自己的模型和领域名词知识库；不论是入组新人还是外部团队都可以通过它快速拉通业务理解

总的来说，通过上面三个层面的可视化展示，我们力图清晰完整地向各方表达业务服务当前的实现逻辑；

通过这种实时清晰的可视方案，业务、开发、测试同学能在研发过程中快速地形成统一语言，提升效率。

3.2.2 服务调试

得益于SDK接入服务的优势，以及上面的链路大图；

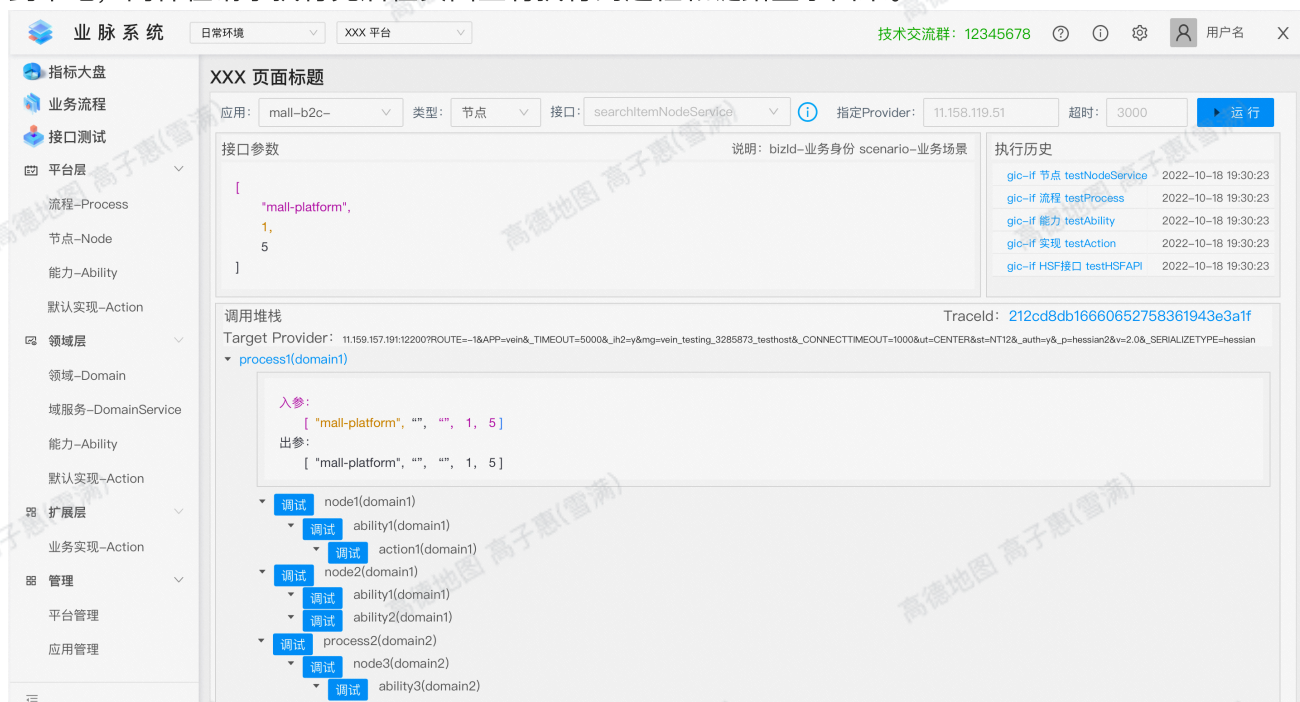
我们也做了对服务的接口调试功能，通过对服务中任意位置的在线调试

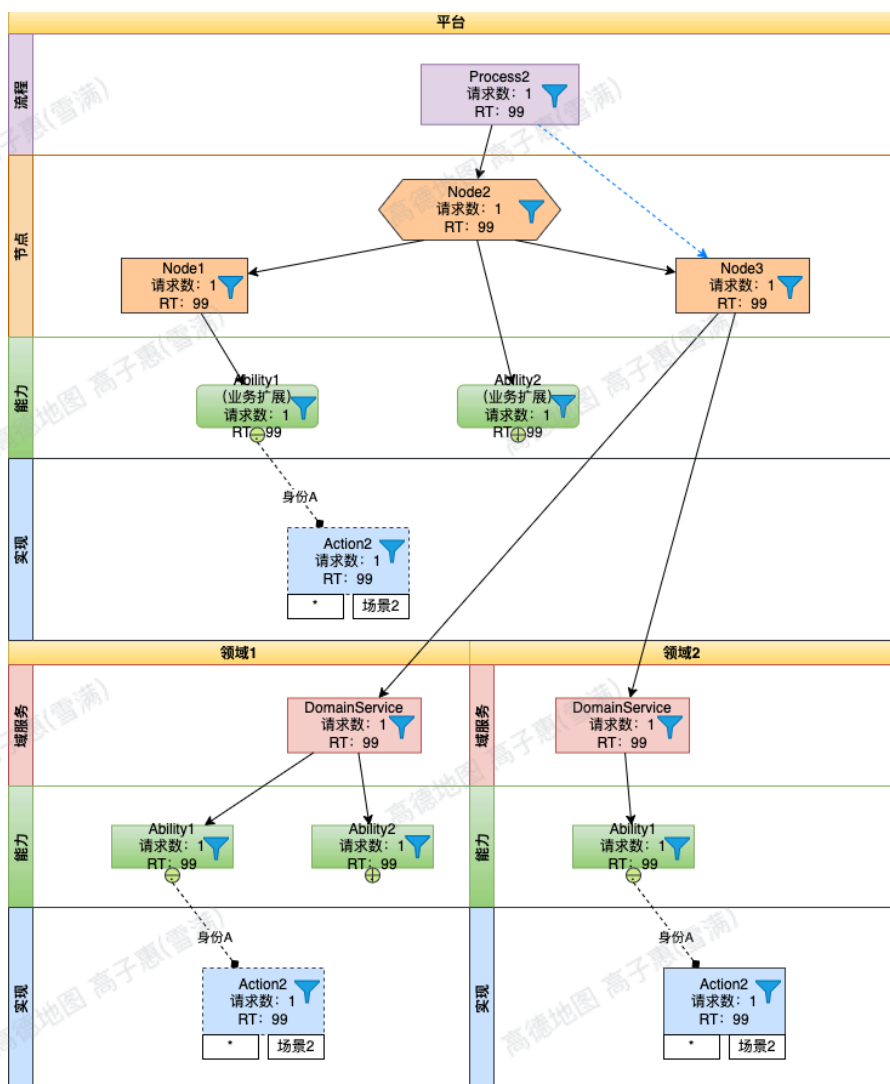
1) 我们可以清楚地了解到当前入参下服务的执行过程，这个过程包括：

- 各层节点的调用堆栈、出入参、性能消耗
- 执行过程的traceld
- 执行过程的实际链路

2) 同时，在返回的结果中我们也可以在任意位置点击调试，将返回的入参再次带入，进行连续调试；

3) 如果我们想对本地正在开发的服务进行调试，我们也可以在Provider处指定自己的ip，请求会直接到本地，同样在请求执行完后在页面上将执行的过程和链路显示出来。





3.2.3 运行管控

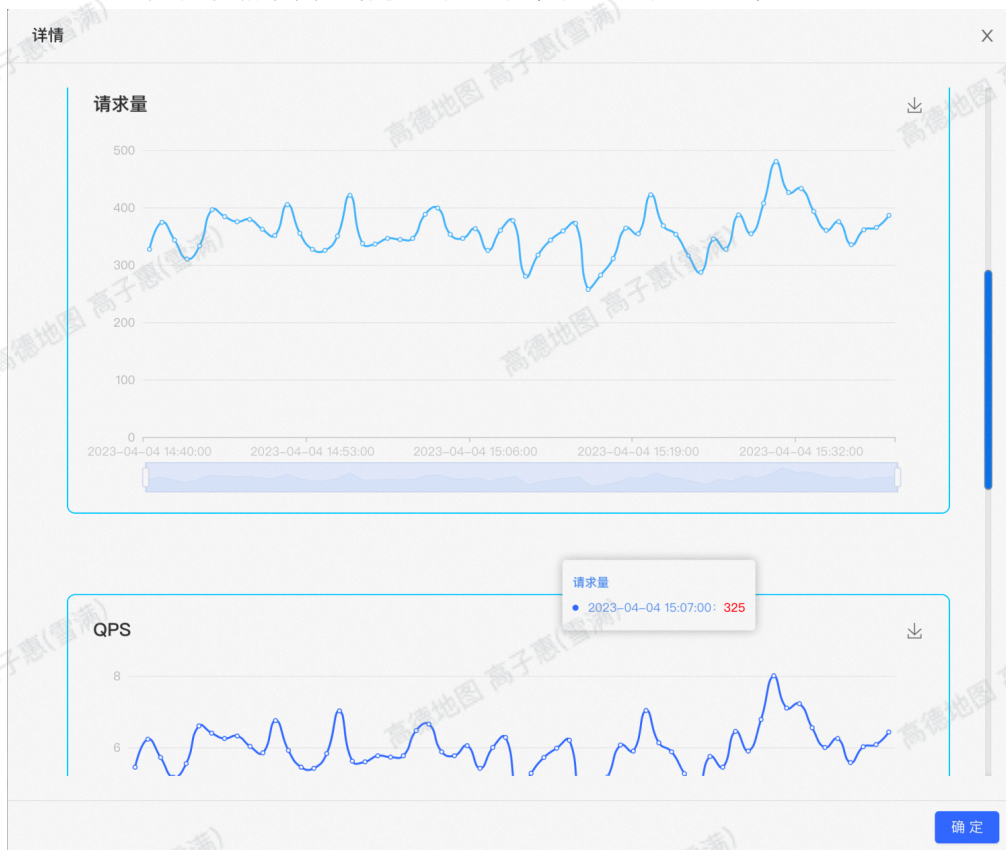
业脉平台的运行管控分为两个方面：

1) 首先是对服务的运行指标监控

- 基于业务身份 维度的请求数、成功率监控

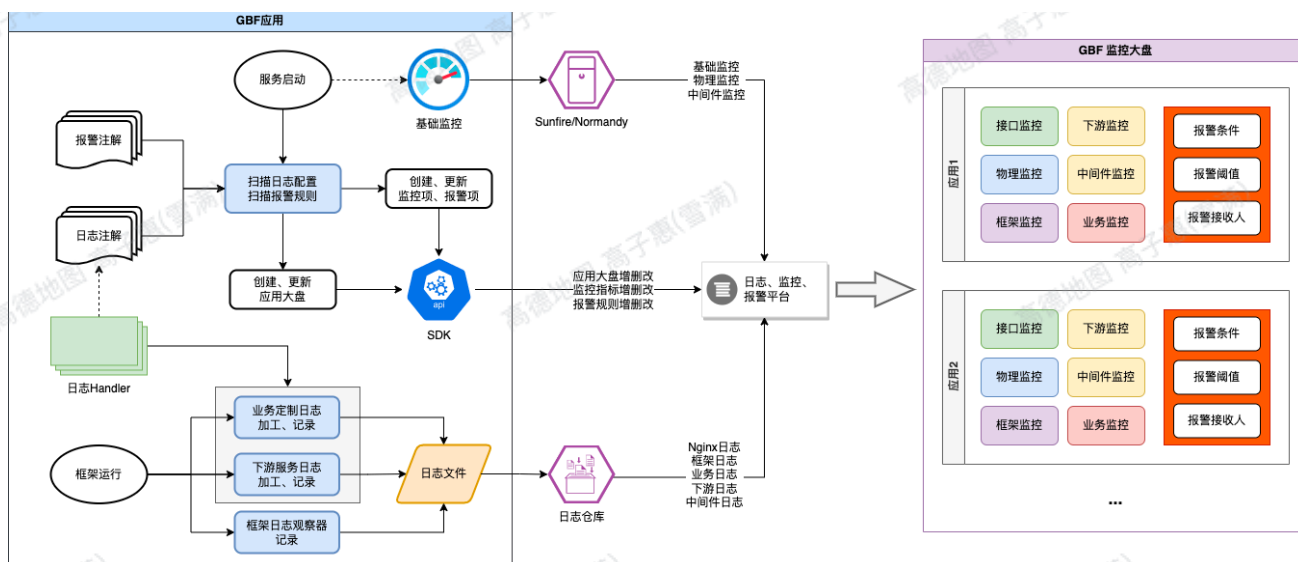


- 基于流程、节点维度的请求数、成功率、QPS、RT统计



2) 其次是对服务稳定性的过程管控

- 当前框架对运行中的管控主要手段为框架运行日志以及异常日志的收集打印
- 目前也正在与稳定性平台合作，对稳定性能力做进一步迭代升级，包括



- 接入框架的应用自动创建监控大盘，并具备通用的基本监控指标和框架运行指标的监控报警能力
- 用户可灵活定制自己的监控指标：接口指标、业务指标、下游指标
- 用户可灵活定制自己的报警规则：指标、规则、阈值、接收人

3.3 提效组件

同时我们也在持续推出用于研发提效的组件包，目前已有规则扩展（diamond）、fetcher客户端（http、hsf）、缓存Cache（session、BizMetrics）、线程池等组件

3.3.1 规则扩展

规则扩展作为框架的能力扩展中一个重要的组成部分，我们将它以扩展包的形式发布出来；需要的项目可以单独引用来提升应用的扩展能力，利用灵活强大的规则系统我们可以做到：

- 提供规则逻辑的可视化能力，便于业务同学理解
- 提供基于可视化的维护能力，将扩展维护职责转移到专业人员
- 提供业务规则的配置维护平台，降低规则扩展的发布成本，提升稳定性、时效性
- 提供基于规则的复用能力，能够将相同业务规则统一复用

为了实现以上目标，我们调研了诸多优秀的框架和规则引擎，规划了适合GBF及部门业务的规则扩展系统

1. 框架中的规则配置

我们在框架中实现了基于注解配置的规则扩展能力，并且规则的扩展可灵活定义和使用；首先，我们可以在框架的扩展实现（Action）处不写代码而直接配置对应规则来实现特性业务定制

复制代码

```
@Override
@Action(name = "banner信息扩展-默认实现")
@ConfigExtension(dataId = "${gbf.extension.banner.data-id}", groupId =
"${gbf.extension.banner.group-id}",
```

```

        receiver = BannerReceiver.class, calculator =
BannerFirstCalculator.class)
    public abstract BannerInfoConfigDO parseBannerInfo(OperationParam param);

```

其次，我们也可以直接在框架中任意位置通过注解修饰声明的字段来随时接入某个规则配置实现特性业务定制

复制代码

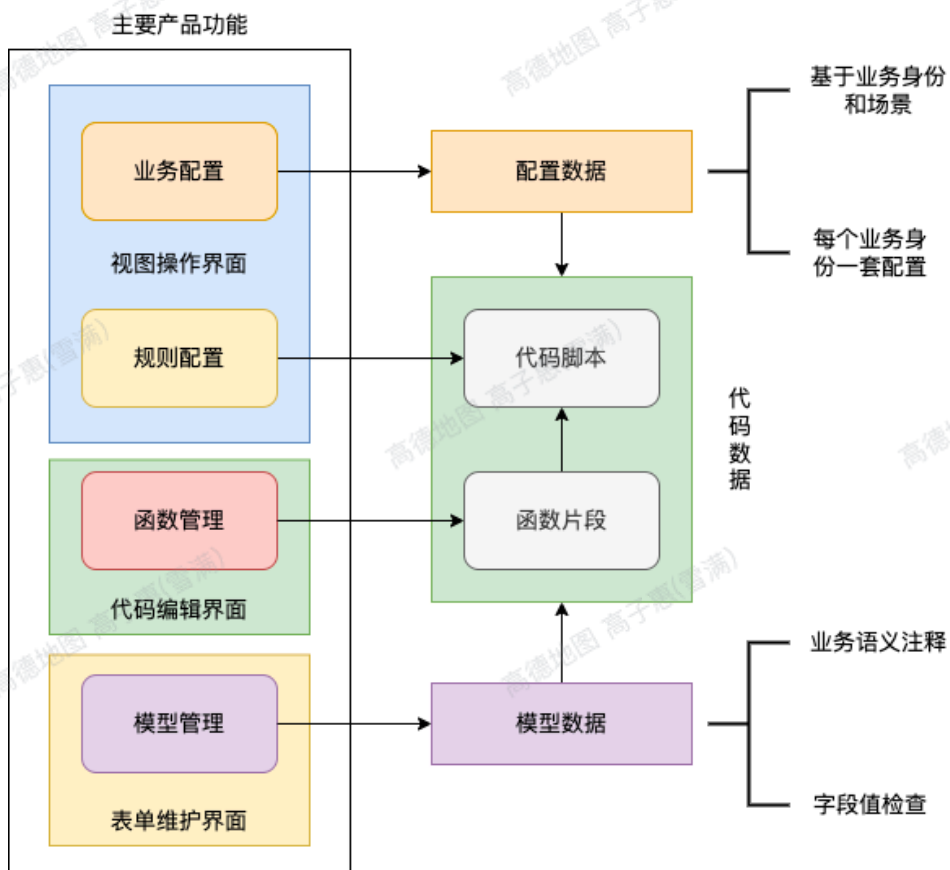
```

@ConfigExtension(dataId = "${gbf.extension.banner.data-id}", groupId =
"${gbf.extension.banner.group-id}",
    receiver = BannerReceiver.class, calculator =
BannerFirstCalculator.class)
protected RuleProvider<DynamicCondConfig<BannerInfoConfigDO>,
BannerInfoConfigDO> bannerConfigList;

```

2. 产品侧的规则维护

目前我们的规则已经通过移植MIF框架的包实现了对US运营平台的无缝对接，现有运营配置可平滑迁移到使用GBF框架的业务中；



同时我们未来也将发展更多规则管控能力：

- 可视化业务策略配置规则
- 简单逻辑的代码脚本规则
- 可复用的脚本函数管理

d.统一的出入参模型管理

e.规则扩展的调试能力

相信届时规则系统将会成为GBF配套组件中的提效神器。

3.3.2 Fetcher客户端

接入GBF前我们的业务服务在访问下游时，虽然能规范地规划出repo模块作统一的访问收口；但在http客户端、hsf调用方式以及对下游的访问管控并没有一个统一的方案；

这导致了不同服务不同接口在稳定性保障方面乏力；

为了解决这个痛点，我们在改造US服务的过程中也推出了基于接口注解方式的[下游客户端Client](#)它们具有如下特点：

1. 使用简单，基于注解灵活配置

a. http

注解使用：

▼ httpFetcher

复制代码

```
@HttpFetcher(name = "poiHttpFetcher")
public interface PoiHttpFetcher {

    @HttpFetcherMethod(name = "get")
    String get(String id);

}
```

参数配置：

▼

复制代码

```
fetcher:
  http:
    configs:
      poiHttpFetcher:
        host: 127.0.0.1
        port: 7001
        use-vip-server: false
        timeout: 1000
        methods:
          get:
            path: /poi/get
            decoder:
com.amap.gbf.demo.repository.impl.fetcher.decoder.StringDecoder
```

b. hsf

注解使用：

[复制代码](#)

```
@HSFFetcher(name = "productHsfFetcher")
public interface ProductHsfFetcher extends ProductServer {

    @HSFMethod(name = "get")
    String get(String id);

}
```

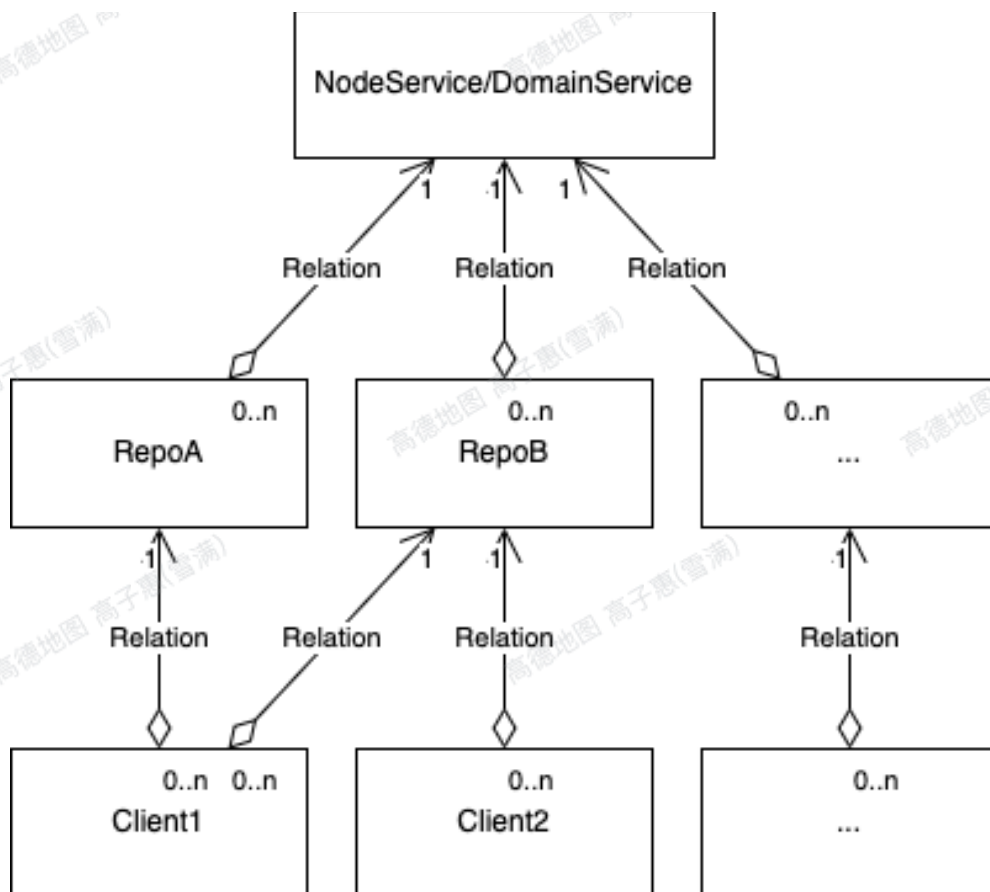
参数配置：

[复制代码](#)

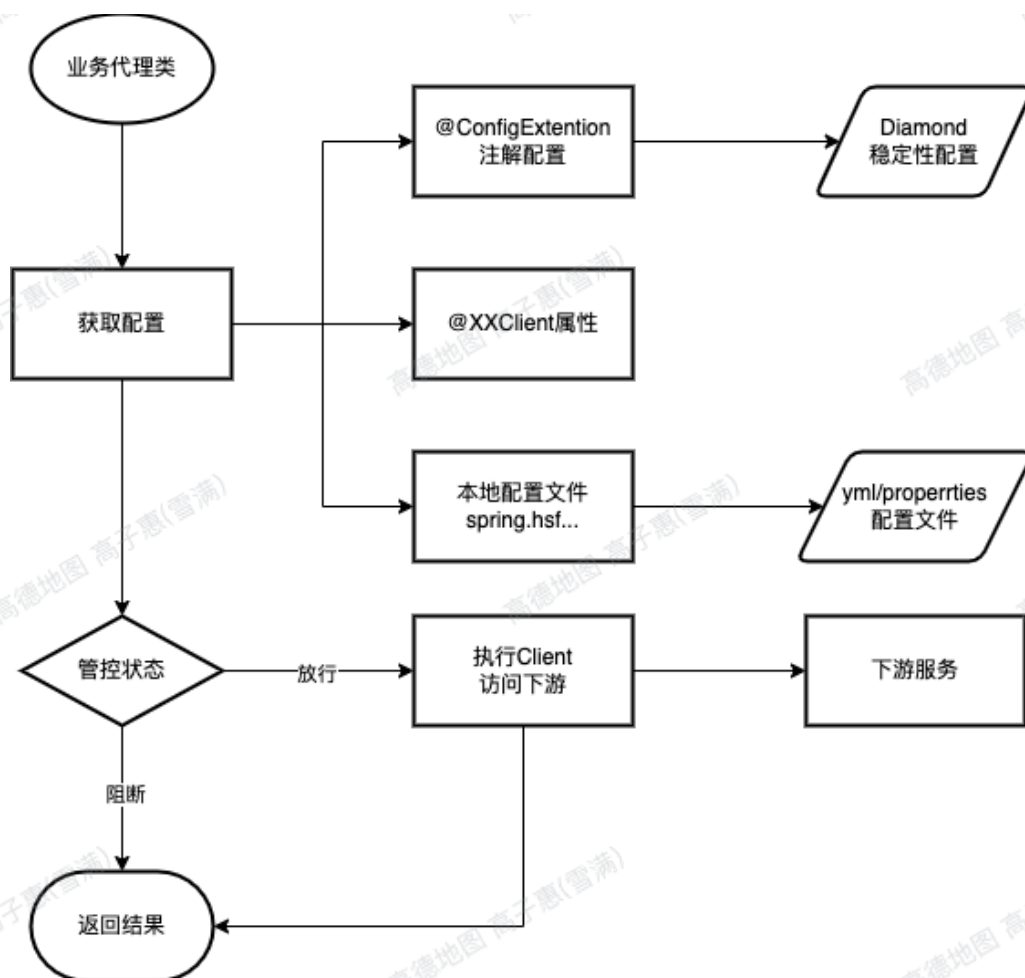
```
fetcher:
  hsf:
    group: gbf-demo
    configs:
      productHsfFetcher:
        version: 1.0.0
        timeout: 1000
```

此外：hsfFetcher支持无Interface类的泛型调用，只需在配置中指定InterfaceName即可。

2. 数据能收容，repo层的数据可收容进GBF，在配套的业脉平台可视化展示



3. 最后在运行期可通过灵活的配置来实现缓存、超时、限流、熔断、降级、预案等管控能力，保障稳定性



3.3.3 流控能力

在2.3.7版本中GBF依托于集团的Sentinel中间件实现了流控能力，进一步提升框架的容灾预案能力；

- 具备了入口流控、出口流控两个能力；
- 具备了基于业务身份、场景、测试标、压测标、**Process**、**Node...**等层级的热点规则能力
- 具备了基于**Sentinel**的流控能力（授权、系统、集群等）
- 具备了业务定制热点参数的流控能力
- 具备了用户定制**Fallback**处理流控事件的能力

使用条件：

1. 入口流控
 - a. 引入gbf-component-common包
 - b. 在需要管控的入口方法上加@RequestHandler注解
 - c. 登录Sentinel控制台创建流控规则
2. 出口流控
 - a. 引入gbf-component-fetcher包
 - b. 使用fetcher作为出口流量的客户端
 - c. 登录Sentinel控制台创建流控规则

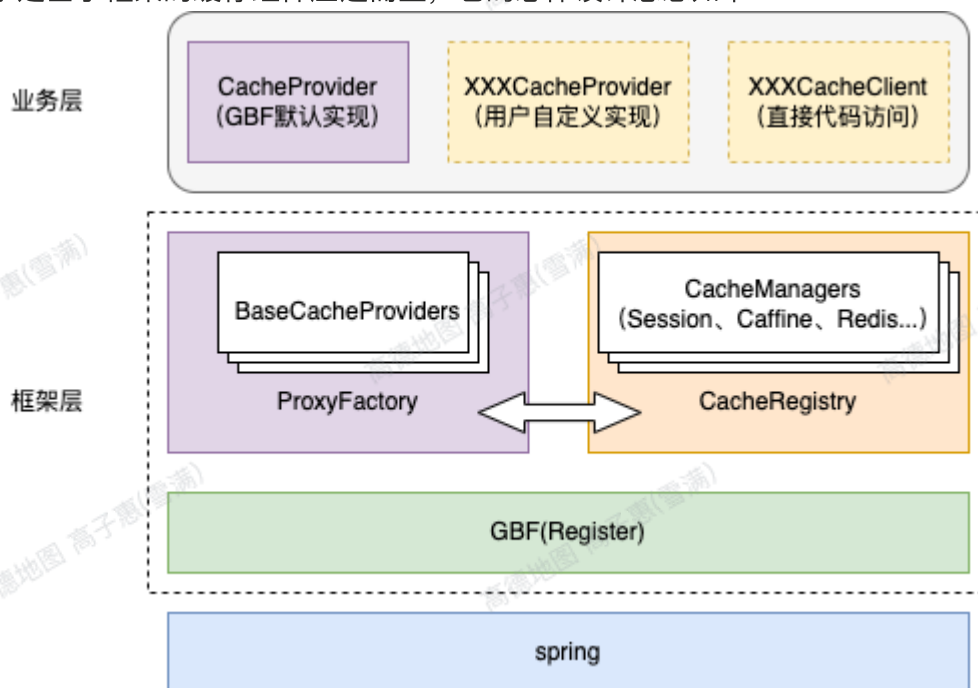
详细流控能力使用手册见：[GBF流控功能使用手册](#)

3.3.4 缓存能力

随着业务的深入迭代，我们的服务也面临着下面两种

- 业务中出现一个流程（Session）对同一下游Fetcher的多次请求调用
 - 参数一样，数据也一样
 - 在流程中无法收敛为一次执行
 - 期望避免流量放大
 - 能够加速整个流程的执行
- 一些常用不变的数据期望长时间缓存在内存中
 - 减少下游访问
 - 执行过程提速
 - 地区列表，邮编等数据

于是基于框架的缓存组件应运而生，它的总体设计思想如下：



主要有下面几点：

- 缓存功能的使用基于GBF现有的生命周期进行实现
 - Register启动扫描包初始化注解的字段对应的Provider实例
 - 创建对应的Registry存储所有缓存Manager以及其下的各种缓存实例
 - 通过实例化后的Provider对象赋值到声明字段来让业务声明后直接用
- 缓存的分层参照JSR107规范进行设计
 - Provider：业务定义和调用的关联点—(可扩展实现多缓存组合使用功能)—
 - Manager：缓存实例管理器（包含指标统计功能、Session回调销毁等）
 - Cache：缓存实例和缓存管理器（实现了对三方缓存功能的封装）
- 面向接口和扩展设计，业务用户在每一层均可实现基于自有逻辑的定制实现

同时，缓存的使用也基于简单的注解实现：

1. 引用gbf-component-cache包
2. 定义缓存实例

[复制代码](#)

```
/**
 * Session级，一次请求执行过程内有效
 */
@SessionCache
protected CacheProvider<String, ProductPO> productCache;
```

3. 代码中使用

[复制代码](#)

```
return productCache.get(productId,
    k ->
    ProductPO.builder().productId(productId).productName(productHsfFetcher.get(productId)).build());
```

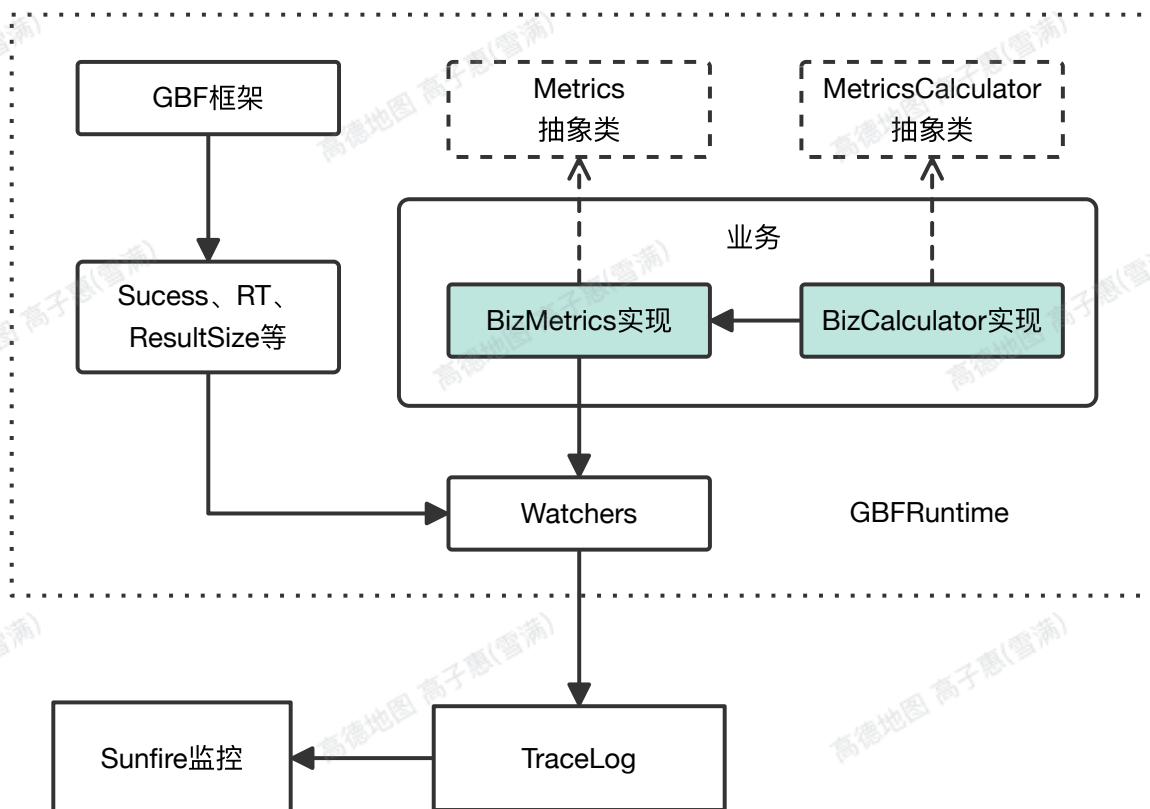
详情参见：[GBF缓存功能](#)

3.3.5 业务指标

在上文业脉的运行管控中也提到了日志和监控指标；但是框架所能提供的日志和指标方案必然是通用的；

当我们的业务有自己的特殊指标和监控诉求时该如何解决呢？

我们经过用户需求调研后，推出了自定业务指标组件包，其运作原理如下：



业务如果需要在Trace中打印指标需要定义两个东西：

- 业务指标类：扩展自com.amap.mall.gbf.client.metrics.Metrics



复制代码

```

@EqualsAndHashCode(callSuper = true)
@Data
@Builder
public class SimpleMetrics extends Metrics {
    public Integer count;
}

```

- 业务指标计算器：扩展自com.amap.mall.gbf.client.metrics.MetricsCalculator



复制代码

```

@Component
@Scope("singleton")
public class SimpleMetricsCalculator extends MetricsCalculator<SimpleMetrics> {

    @Override
    public SimpleMetrics onComplete(final Object[] args, final Object result) {
        return SimpleMetrics.builder().count(ObjectUtil.count(result)).build();
    }
}

```

然后在需要计算指标的位置设置计算器即可：

此外，如果我们的计算指标需要在前面代码中提前计算并传递到计算器，我们可以用缓存中的 MetricsData；

- ### 3. 指标计算器中读取数据

23 / 34

```

@Component
@Scope("singleton")
public class TestMetricsCalculator extends SimpleMetricsCalculator {
    @Override
    public SimpleMetrics complete(Object[] args, Object result) {
        System.out.println("TestMetricsCalculator.onComplete");
        System.out.println("TestMetricsData:" + getMetricsData().get("test"));
        return super.complete(args, result);
    }

    @Override
    public SimpleMetrics interrupt(Object[] args, Throwable throwable) {
        System.out.println("TestMetricsCalculator.onInterrupt");
        System.out.println("TestMetricsData:" + getMetricsData().get("test"));
        return super.interrupt(args, throwable);
    }
}

```

注：

- **MetricsData**基于**Session**缓存实现，最大淘汰时间为5s；原理详情见缓存设计文档（[GBF缓存功能](#)）
- 出于合理使用的考虑，**MetricsData**限制可在任意位置写入删改指标数据，只能在**MetricsCalculator**中读取
- 详细使用文档见：[GBF业务指标使用手册](#)

more

其他组件不再一一赘述

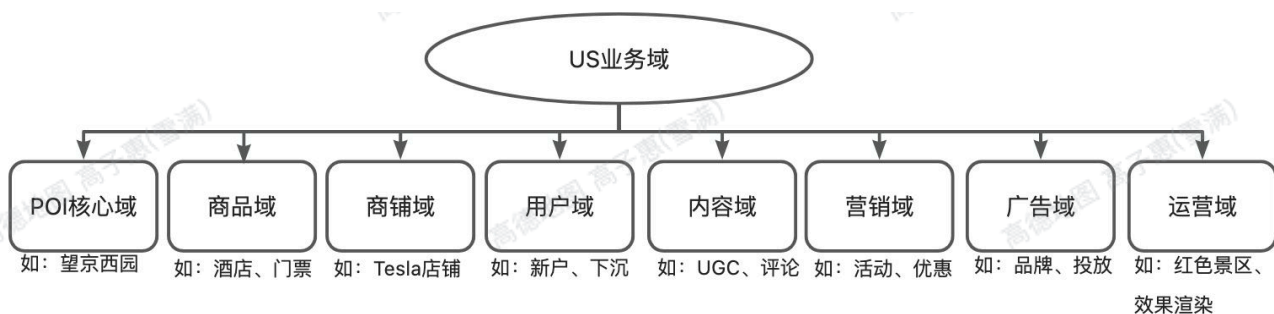
- 线程池组件：gbf-component-threadpool
- ...

4. 建设案例

以[US的业务接入](#)过程为例，我们经历了如下过程：

4.1 领域划分

接上面的架构思想，我们要接入业务首先要做的就是梳理业务的领域并将其清晰地划分出来，例如US的业务域我们经过讨论和调整，最终达成如下划分方案：



同时，我们也要明白“唯一不变的是变化”这个道理，随着未来业务的发展，复杂度提升到必要程度时，我们也可以对现有的划分进行拆分升级；

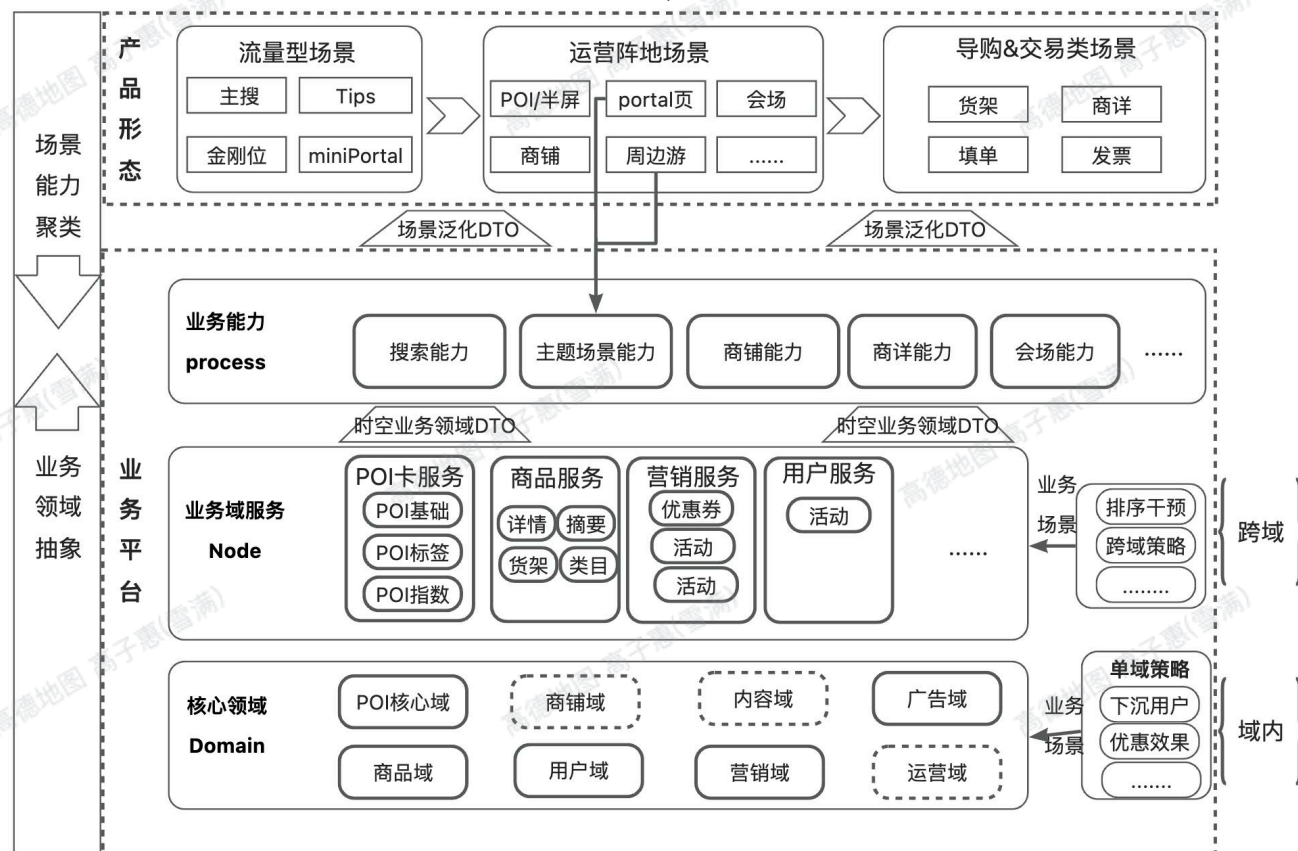
但这些修改变更并不是随意进行，域的划分和后续的迭代需始终遵守一个统一的领域划分指导原则：

如[US领域划分指导原则](#)

4.2 业务架构

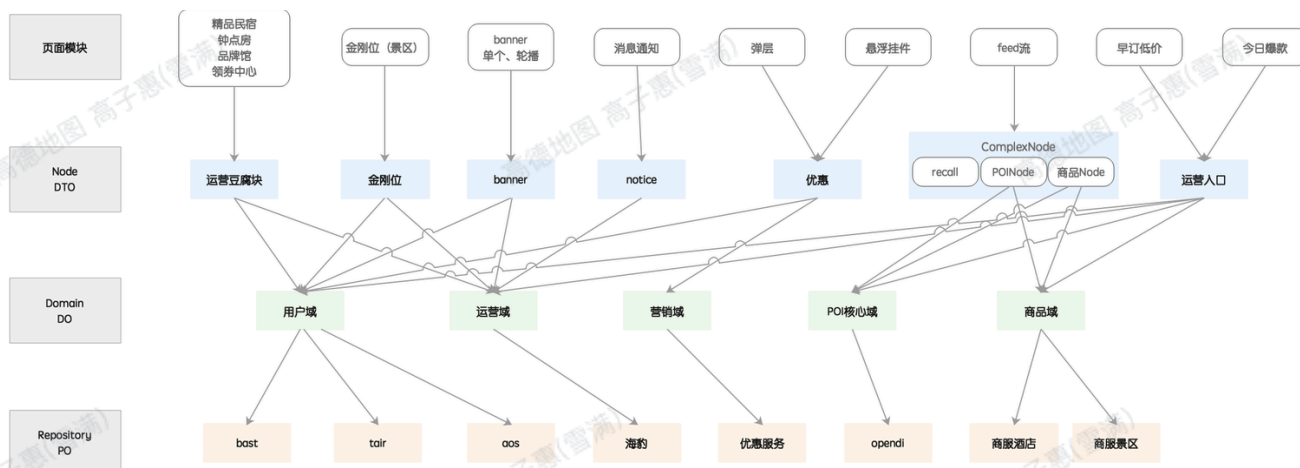
对US的领域划分清楚的同时，我们也能清楚的看到：**US的业务本质就是数据+策略**

因此我们基于上面划分的领域以及对US业务的理解，梳理出了业务架构图：



从这个架构图我们可以清晰地得出系统划分、领域定义、业务分层结果

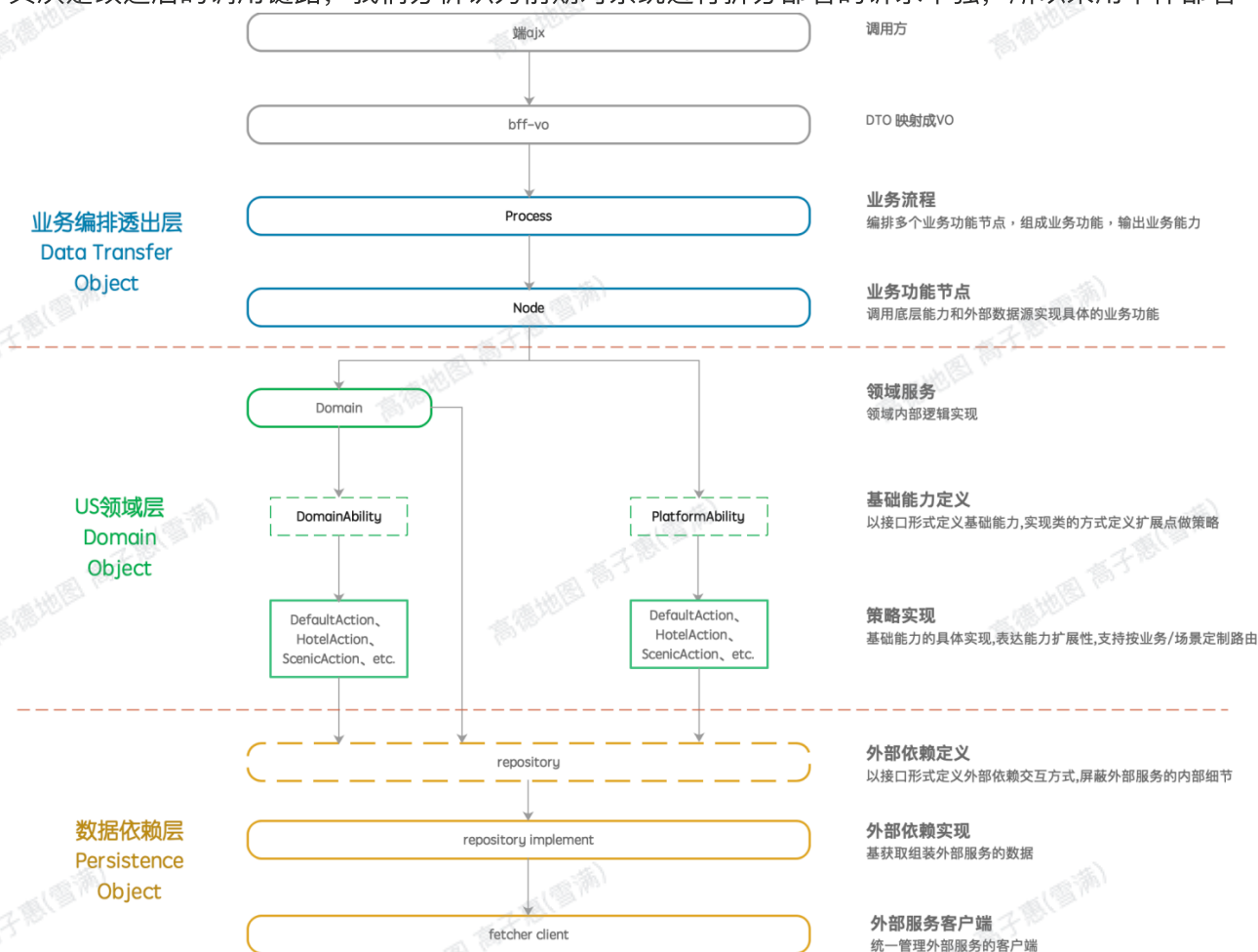
4.3 标准建设



标准能力建设不是为了还原现在不合理的调用链路，而是通过统一的 领域建设 完成业务架构分层 和 统一技术架构，实现清晰合理的调用关系，提升工程效率。

4.4 调用链路

其次是改造后的调用链路，我们分析认为前期对系统进行拆分部署的诉求不强，所以采用单体部署：



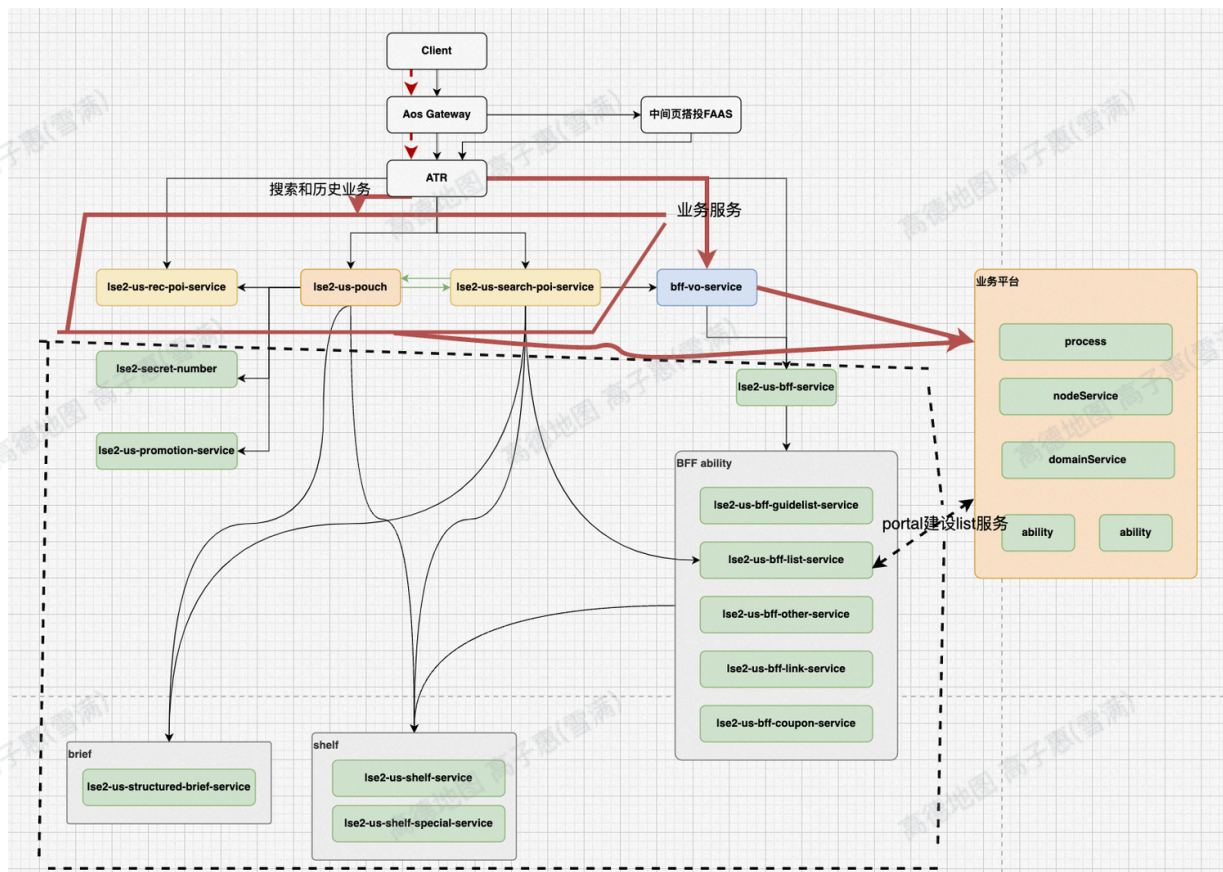
未来可依据域服务的**复杂度**、**服务分级治理**、**稳定性**需要进行拆分，主要工作及成本：

- 1) 拆包，将需要独立的服务单独打包；
- 2) 本地调用转RPC，当前GBF框架支持设置透出HSF接口的能力；

成本评估：通过提前预研评估，拆分投入较低且风险可控，对业务影响有限。

4.5 系统拆分

而后，如果是从老系统接入GBF；我们需要对原有系统及业务进行拆分，规划出新的系统以及迁移的链路。



US拆分说明：

- 老的java服务通过迁移完成下线
- php持续下沉到新系统
- 由于老版本支持需要，部分老系统只做维护不做迭代

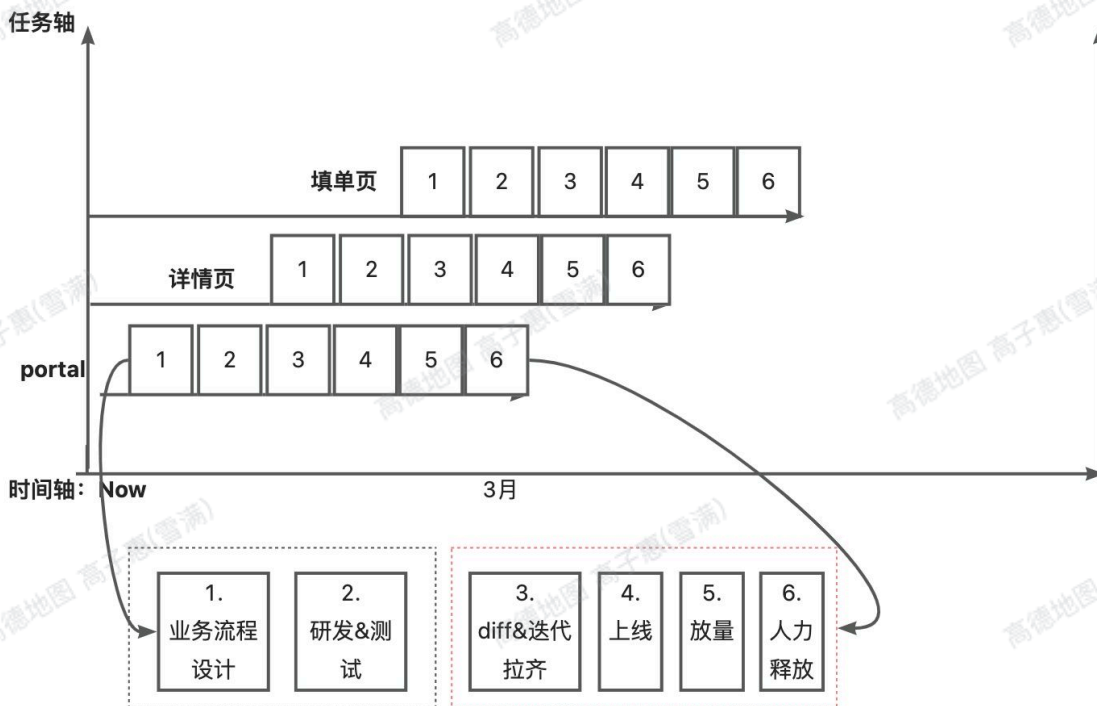
4.6 任务拆解

1) 大节奏：不阻断业务迭代，新、老系统分别开发部署，通过diff明确迁移可迁移状态，按评估节奏进行切换；（参考PHP向JAVA迁移）

说明为什么新方案对老系统的开发成本低（领域能力的复用将开发工作内聚到领域内，平台开发和业务开发分离解决了研发相互依赖）

2) 对US现有基础模块迁移：通过域能力建设完成对现有模块（list，brief等）的迁移和替换

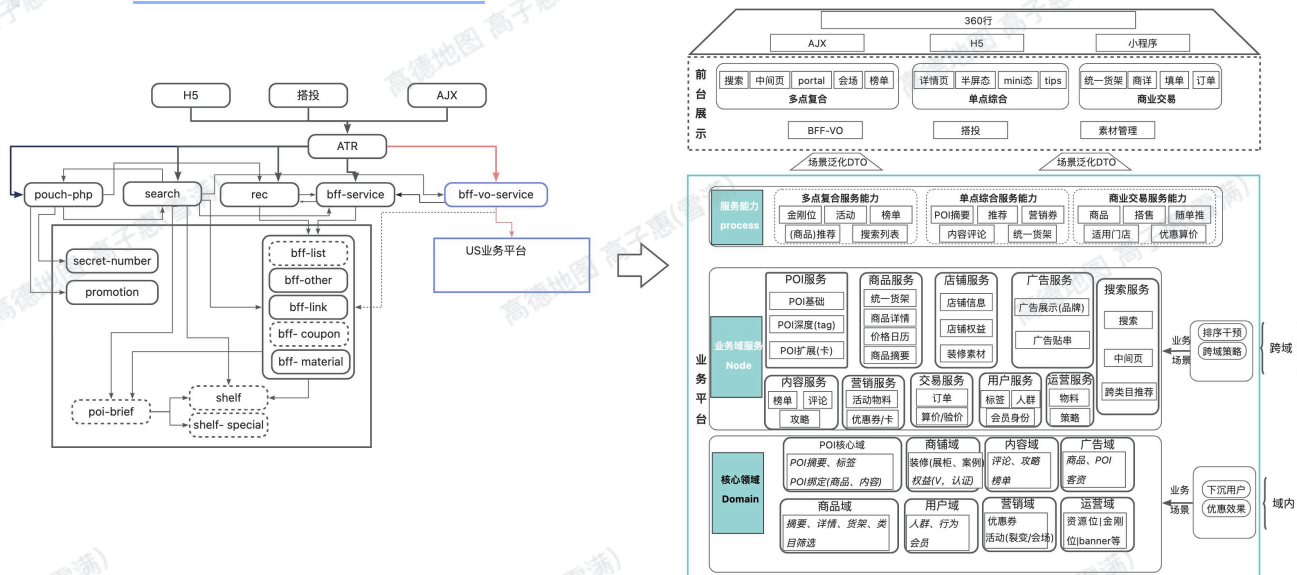
3) 对前端流量入口迁移：按照场景分批次建设和迁移（如portal页，周边游）；



随着前端产品的迁移，后端的领域服务能力建设逐步验证和增强，研发成本逐步降低；

4.7 收益总结

参考：[产研协同提效——项目流程](#)



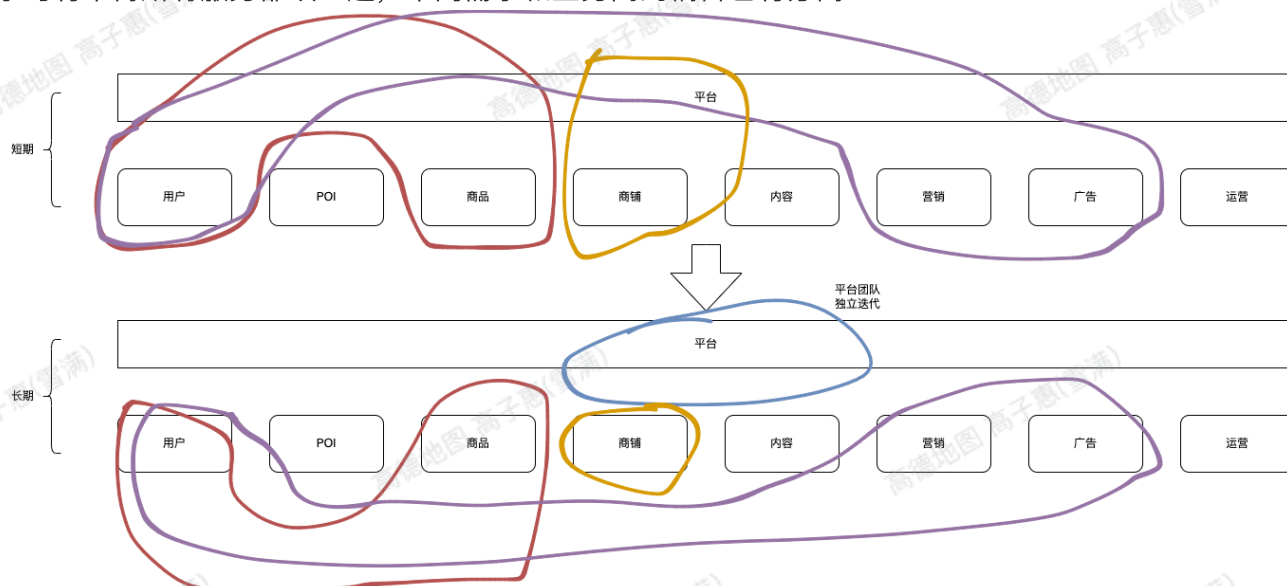
原来US调用链路错综复杂，深度耦合，最长链路6层(Search->VO->service->list->brief->shelf)同时由于US原有的服务和系统划分主要面向客户端和界面，所以每个服务并没有明确的域和业务主题。

而我們再看需求侧的内容，一般都是围绕某个业务主题或领域进行输入：

ID	标题
40512268	用户可协商无责退款
40512236	【用户】新人群一期
40512242	【优惠】酒店新客三单礼
40512260	【转化】新portal页改版
40512249	领券中心
40543671	【民宿】对接人群促销报价
40512254	联动海豹的全链路渲染的资源位配置 【跨版本】
40512272	酒店订单详情页增加飞猪小蜜
40577244	飞猪增加最晚到店时限
40577234	艺龙钟点房增加入参

X	Y	Z	AA	AB	AC
4.19	4.20	4.21	4.22	4.25	4.26
		上线			
		上线			
		上线			
		上线			
					发布

由于我们的系统规划没有按照业务的域来划分，造成了研发过程以及系统运行过程的耦合，在这个过程中产生了[比较明显的效率问题](#)，也使我们的研发节奏绑定较深，往往集中在一段时间扎堆上线。接入GBF后，依托于我们重新梳理的业务逻辑和领域划分，上面的问题便会迎刃而解，我们在研发需求时将不再所有服务都改一遍，不同需求和业务间的耦合也将分离：



如上图接入GBF后，可以得到如下收益

1. 业务的逻辑将会变得收敛集中，相同主题或域的代码将只有一份被不同的业务场景复用
2. 单个业务需求的开发范围变得清晰明确，如基于用户的营销类需求只会变更用户和营销域

3. 域和流程统一后整个流程执行过程的数据模型也将统一，过去做需求花大量时间对协议加字段将得到改善
4. 基于业务和系统间的解耦，我们的任务可以逐渐解绑，做到连续输入，错峰上线，大大提升迭代效率

同时，得益于GBF配套的业脉平台和组件包，我们可以得到更多：

1. 基于SDK上报数据的可视化呈现，提高业务的解释能力，便于业务和开发快速掌握系统的业务逻辑
2. 实时可视的接口调试功能，可以快速精准地调试任意位置的框架节点，并显示执行过程和出入参
3. 使用统一的下游Client扩展包，可以快速定义下游repo层接口，并且全局收容管控
4. 使用规则扩展系统，可以将业务规则与代码分离，将业务规则的维护发布交给专业人员，提升研发效率

经过上述改造后我们在以下几个方面获得了显著的收益：

1. 架构更清晰，链路更短，系统能力大图更明确，领域边界更清晰
2. 代码层次结构清晰，实现了较好的通用能力开放和业务之间的隔离，避免了大量if...else的不良设计
3. 方案简化和统一，各方边界更加清晰，并在过程中完成了大家职责和分工的拉齐
4. GBF框架通过可视化，将代码逻辑透明，辅助研发团队和产品之间快速的信息拉齐，将个人知识转化成共有知识

初步统计接入前后产研数据提效如下：

5. 通过合理的架构设计，实现US系统50%的资源节省，50%的发布效率的提升，应用链路缩短30%
6. 抽象复用建设完成，复用代码率87.5%（通用代码：159295，行业扩展：22682）
7. 新场景支撑提效：50%-80%不等，专项类提升：20%-40%
8. 基于单一服务的业务抽象，使得发布效率相对提升 600%

5. 接入指南

5.1 文档

- 业务落地小白书：[GBF业务落地指导小白书](#)
- 最新使用手册：[GBF使用手册-20230103](#)

5.2 案例

- Demo项目：<https://code.alibaba-inc.com/gaode.arch/gbf-demo>
- US示例项目：<https://code.alibaba-inc.com/gaode.arch/gbf-us-sample>

6. 常见问题

6.1 名词

- Process: 业务流程编排层, 提供面向场景的业务编排能力, 并透出对应接口供外部访问
- NodeService: 流程节点服务, 通过跨领域编排调度提供某一业务主题的节点供流程编排, 或直接被外部访问
- DomainService: 领域服务, 作为领域的主要业务载体和边界, 对外提供域服务接口供节点进行跨域编排调度
- Ability: 扩展点声明, 当业务规则需要根据业务身份进行定制时所用
 - PlatformAbility: 承担平台能力扩展的能力声明
 - DomainAbility: 承担领域能力扩展的能力声明
- Action: 扩展点实现, 继承Ability接口的具体定制实现, 放在平台中的为默认实现 (身份为*), 放在应用扩展中的为业务定制实现 (身份为具体业务身份BizId)
 - PlatformAction: 承担平台能力扩展的定制实现
 - DomainAction: 承担领域能力扩展的定制实现

6.2 使用

Q: 能不能在扩展点实现 (Action) 中访问下游?

A: 通常情况不建议这样使用, 但还是要看访问下游行为, 能否被其他业务身份复用, 假设200个业务身份, 只有一个特殊扩展点实现需要下游系统, 可以在扩展点中访问下游

Q: Node和DomainService所调用的Ability是否一样?

A: 不一样, Node作为平台侧的层级, DomainService作为领域侧的层级, 二者一个是平台业务能力扩展, 一个是领域能力扩展

Q: ComplexNode和Node的区别?

A: ComplexNode作为聚合节点, 是一类特殊的Node节点, 其主要用于对单一业务主题Node的编排复用; 适用于异构体数据的业务形态 (如Feed业务数据), ComplexNode可以调用Ability扩展、下游Repo、以及业务Node, 不可以直接调用域服务 (DomainService)

Q: 领域如何划分?

A: 参见 [US领域设计原则](#)

Q: 身份识别路由失效了

A: 排查方式: 1.检查app包路径被platform包路径覆盖; 2.检查app包提前被启动包覆盖 (@SpringBootApplication(basePackages)); 3.app包没被启动包依赖

Q: DomainService中一定存在扩展点吗?

A: 不一定, 只有出现身份维度的定制需求时, 才使用扩展点

Q: NodeService与DomainService一定是一对一的吗?

A: 不是, NodeService作为业务主题的具体载体, 承担了跨域编排的职责, 因此其与域服务间的关系为一对多

7. 其他

7.1 GBF地址

项目仓库: <https://code.alibaba-inc.com/gaode.mall/gbf>
maven包



复制代码

```
<!-- 框架包 -->
<dependency>
  <groupId>com.amap.mall.gbf</groupId>
  <artifactId>gbf-starter</artifactId>
  <version>${gbf.version}</version>
</dependency>

<!-- 组件包 -->
<dependency>
  <groupId>com.amap.mall.gbf</groupId>
  <artifactId>gbf-components</artifactId>
  <version>${gbf.version}</version>
</dependency>
```

7.2 业脉地址

环境	地址	备注
日常	https://gbf.amap.test/	
预发	https://pre-gbf.amap.com/	
线上	https://gbf.amap.com/	

7.3 交流群

群号: 27215026722



该群属于“本地生活集团”内部群，仅组织内部成员可以加入，如果组织外部人员收到此分享，需要先申请加入该组织。

8. 参考推荐

GBF完整使用手册： [使用手册](#)

GBF整体设计文档： [设计方案](#)

8.1 参考文章

- GBF分层设计及模型： <https://aliyunque.antfin.com/amap-search/nen549/tghr6aibad67ctx?singleDoc#>
- US数据、逻辑分层说明： <https://aliyunque.antfin.com/amap-search/nen549/nbtclcrdistbes70y?singleDoc#>
- US领域设计原则： <https://aliyunque.antfin.com/amap-search/nen549/yg8lx8f3ws742cff?singleDoc#>
- US业务平台建设： <https://aliyunque.antfin.com/amap-search/gll2zw/atxrl5bu7o9y8ht?singleDoc#>
- US使用GBF建设portal实战： <https://aliyunque.antfin.com/amap-search/gll2zw/sx308gm4fvi78ugp?singleDoc#>
- GBF在US建设标准业务能力： <https://aliyunque.antfin.com/amap-search/gll2zw/yxgvp9g6vk0s0o3h?singleDoc#>

- GBF改USPortal页迁移过渡方案: <https://aliyuque.antfin.com/amap-search/gll2zw/xbrumwodoq5oopx1?singleDoc#>
- US酒旅服务发布问题分析: <https://aliyuque.antfin.com/amap-search/nen549/vydp7wdsbcmpbo28?singleDoc#>
- US效率问题整理: <https://aliyuque.antfin.com/amap-search/nen549/lr7n57e1t3xmle0?singleDoc#>
- US业务功能梳理 (酒店portal) : <https://aliyuque.antfin.com/amap-search/gll2zw/aacuhkqczwu8g464?singleDoc#>
- 星环研究总结: <https://aliyuque.antfin.com/amap-search/nen549/bvwo0me1hr98h027?singleDoc#>
- US的个人架构思考: <https://aliyuque.antfin.com/amap-search/nen549/mzvzc5pezpd8n5u5?singleDoc#>

8.2 推荐文章

- 业务系统架构实践总结: <https://ata.alibaba-inc.com/articles/243207>
- DDD领域驱动设计: 知识点与思考: <https://ata.alibaba-inc.com/articles/178459#ikoe-aiU>
- 星环入门学习指引: <https://ata.alibaba-inc.com/articles/143837>